

Kalshi Crypto Trading Bot — Investor Whitepaper

Gabriel Kagan

February 2026

Contents

Executive Summary	2
The Opportunity	2
Why Prediction Markets?	2
Why Kalshi?	2
The Systematic Edge	3
Strategy Overview	3
1. Observe	3
2. Estimate	3
3. Filter	4
4. Size	4
5. Execute	4
Edge Sources	4
Speed and Breadth of Information	5
Volatility Sophistication	5
Distribution Fitting	5
Fee Optimization	5
Risk Management	5
Conservative Position Sizing	5
Automatic De-Risking	6
Multi-Asset Trading	6
Multiple Safety Checks	6
Hard Price Boundaries	6
Performance and Observations	6
Market Evaluation Summary	6
Settled Outcomes	7
Win Rate by Entry Price	7
Infrastructure and Reliability	7
Always-On Operation	7
Continuous Deployment	7
Complete Audit Trail	8

Crash Recovery	8
Technical Appendix	8

Executive Summary

This document describes an automated trading system for **Kalshi**, the first CFTC-regulated prediction market exchange in the United States. The bot trades short-duration cryptocurrency contracts — binary options that settle every 15 minutes — using a systematic, model-driven approach designed to capture small but frequent mispricings.

The system monitors real-time prices across seven data sources, estimates outcome probabilities using institutional-grade statistical models with per-asset distribution fitting, and executes trades only when it identifies a clear edge over the market price. Every aspect of the strategy — from market selection to position sizing to execution — is designed around capital preservation and disciplined risk management.

Key facts:

- 0 markets evaluated during the observation period (N/A)
 - 0 markets settled with verified outcomes
 - N/A observed win rate across all settled positions
 - Fully automated, always-on operation with complete audit trail
 - Live trading with real capital since February 2026
-

The Opportunity

Why Prediction Markets?

Prediction markets allow participants to trade contracts on the outcomes of real-world events. Unlike traditional financial markets, where pricing depends on complex fundamental analysis, prediction market contracts resolve to a simple binary outcome: the event either happens or it doesn't. This clarity creates a well-defined mathematical edge for quantitative approaches.

Why Kalshi?

Kalshi is the only CFTC-regulated prediction market exchange in the US, providing the legal and structural protections of a regulated financial venue. Its crypto contracts offer several attractive properties for systematic trading:

- **High frequency:** New contracts launch every 15 minutes, 24/7, across four major cryptocurrencies (BTC, ETH, SOL, XRP)
- **Short duration:** Each contract settles within 15 minutes, meaning capital is never locked up for long
- **Binary outcomes:** Contracts pay exactly \$1 if the asset stays above a threshold, \$0 otherwise — no partial outcomes or complex payoffs
- **Hundreds of opportunities daily:** The combination of four assets, multiple strike prices, and 15-minute windows creates a deep pool of tradeable markets every day

The Systematic Edge

Retail traders on Kalshi typically check one price source and make intuitive judgments about whether a crypto asset will stay above a given level. This approach is slow, inconsistent, and unable to process the volume of markets available.

A systematic approach has structural advantages: it can process every market, every window, with consistent discipline — never skipping a promising trade due to fatigue, and never chasing a bad one due to emotion.

Strategy Overview

The bot follows a disciplined five-step process for every 15-minute trading window:

1. Observe

The system maintains real-time connections to seven data sources simultaneously:

- **Four spot exchanges** (Coinbase, Binance, Kraken, Bybit) — providing live cryptocurrency prices with sub-second updates
- **Deribit** — the leading crypto derivatives exchange, providing implied volatility data that reflects the market’s forward-looking risk expectations
- **CoinGlass** — providing funding rate data from perpetual futures markets, indicating leverage buildup and crowded positioning
- **Kalshi** — the prediction market itself, providing current contract prices, orderbook depth, and real-time fill notifications via WebSocket

This multi-source approach means the bot sees price movements developing across global markets before they are reflected in Kalshi contract prices.

2. Estimate

For every active market, the bot estimates the probability that the underlying asset will stay above the contract’s threshold for the remainder of the 15-minute window. This estimate incorporates:

- Current spot price relative to the threshold
- Recent realized volatility (how much the price has been moving), estimated using noise-robust academic methods
- Options-implied volatility (what the derivatives market expects)
- Cross-exchange price signals (whether other exchanges are leading a move)
- Funding rate regime (whether leveraged positions suggest mean-reversion risk)

The probability model uses **Normal Inverse Gaussian (NIG) distributions** fitted specifically to each cryptocurrency’s return characteristics. Unlike generic models, NIG captures both the heavy tails (large moves are more common than a bell curve predicts) and the asymmetry (upward and downward moves have different frequencies) unique to each asset.

3. Filter

Most markets are not worth trading. The system applies a rigorous multi-stage filter that rejects markets for any of the following reasons:

- The model's estimated probability is too low (the contract is unlikely to pay out)
- The Kalshi price is too high (not enough profit potential) or too low (too much uncertainty)
- The edge after fees is insufficient — must exceed 1% after taker fees (evaluated at worst-case rates)
- The model and the market disagree by a suspicious margin (suggesting the model may be missing information)
- Statistical inputs appear unreliable (extreme z-scores indicating potential data issues)

Of 0 markets evaluated during the observation period, the vast majority are correctly identified as unprofitable and filtered out — the system is highly selective.

4. Size

For the small number of markets that pass all filters, the bot determines the appropriate position size using a conservative mathematical framework (a fractional Kelly criterion). Position sizes are:

- **Small by design** — never risking more than a carefully calculated fraction of the bankroll
- **Automatically reduced during drawdowns** — if the account balance drops, position sizes shrink proportionally
- **Hard-capped** — regardless of model confidence, no single trade can exceed the maximum position limit

5. Execute

The bot uses a fee-minimizing execution strategy with intelligent escalation:

- **Maker-first:** It initially places limit orders with `post_only` guarantees, earning 75% lower fees than aggressive orders
- **Three-tier rejection handling:** If a maker order is rejected (locked spread), the system tries a degraded maker (worse price), then escalates to an aggressive taker order — but only after re-verifying the trade is still profitable at the higher fee rate
- **Real-time fill detection:** Kalshi WebSocket provides instant fill notifications at zero API cost, with REST polling as a backup
- **Smart escalation:** If a limit order hasn't filled as the window nears expiry, the bot first tries to amend the order in-place (faster than canceling and re-placing), then falls back to immediate-or-cancel taker orders
- **Price re-validation:** Before every execution step, the bot re-checks current market conditions to confirm the trade still makes sense

Edge Sources

The bot's expected profitability comes from four structural advantages:

Speed and Breadth of Information

While a typical Kalshi trader might check the Bitcoin price on one website, this system simultaneously processes real-time data from seven professional-grade feeds. It detects cross-exchange price movements — where a large buy on Binance precedes a move on Coinbase — and incorporates these signals before the Kalshi market adjusts.

Volatility Sophistication

The probability of a crypto asset staying above a given price depends critically on how much the price is expected to move. The bot uses academic-grade volatility estimation techniques (Realized Kernel estimators from Barndorff-Nielsen 2008, data-adaptive bandwidth selection, Mincer-Zarnowitz R^2 -weighted blending) that are standard in institutional finance but rare among retail prediction market participants. This produces more accurate probability estimates, especially during volatile periods.

Distribution Fitting

Most quantitative models assume returns follow a simple bell curve (Gaussian) or a generic fat-tailed distribution. This system fits **Normal Inverse Gaussian distributions** to each cryptocurrency individually, capturing the specific tail behavior and asymmetry of BTC, ETH, SOL, and XRP. This produces measurably better probability estimates — statistical tests confirm NIG fits the actual data far better than generic alternatives.

Fee Optimization

Kalshi charges different fees for different order types. The bot’s maker-first execution strategy with three-tier escalation captures the 75% fee discount available to limit orders whenever possible, directly improving the profit margin on every trade. When forced to pay taker fees (locked spreads), the system re-verifies profitability before proceeding. This seemingly small advantage compounds significantly over hundreds of trades.

Risk Management

Capital preservation is the system’s primary objective. Every design decision prioritizes survival over aggression.

Conservative Position Sizing

The bot uses an **edge-tiered** approach — higher-conviction trades (greater model edge over the market) receive larger allocations, while lower-edge trades get minimal sizing. The largest tier risks 50% of bankroll only on trades with 4%+ fee-adjusted edge, and the smallest tier risks just 10%. In practical terms:

- Individual trades risk a precisely calculated fraction of the bankroll, proportional to estimated edge
- Even a string of losses has a limited impact on total capital
- The sizing tiers are calibrated against actual trade performance data

Automatic De-Risking

If the account balance drops below certain thresholds relative to its starting value, position sizes are automatically reduced:

- At **90% of starting balance**, sizes are cut in half
- At **80% of starting balance**, sizes are cut to one-quarter

This creates a geometric de-risking curve: the more the account loses, the less it risks, making recovery from drawdowns more manageable.

Multi-Asset Trading

The bot can trade multiple assets per 15-minute window, concentrating capital on the best available opportunities while maintaining independent risk assessment for each position.

Multiple Safety Checks

Before any trade is placed, the system verifies:

- The model's probability estimate passes a sanity check against the market price
- The estimated edge exceeds the minimum threshold after accounting for all fees (at worst-case taker rates)
- The contract price falls within acceptable bounds (not too cheap, not too expensive)
- No extreme statistical indicators suggest unreliable model inputs
- The position size respects all hard limits and drawdown adjustments

If any single check fails, the trade is refused — no exceptions. The system is designed to say “no” far more often than “yes.”

Hard Price Boundaries

The bot only trades contracts priced between 86 and 99 cents. Below 86 cents, historical data shows poor win rates and excessive uncertainty. Above 99 cents, the potential profit is too small to justify the risk. This guardrail eliminates an entire class of low-quality trades.

Performance and Observations

This section contains live statistics from the bot's observation database, updated on each whitepaper build.

Market Evaluation Summary

During the observation period (N/A), the bot evaluated **0** individual markets across BTC, ETH, SOL, XRP. The vast majority were correctly identified as unprofitable and filtered out at various stages:

Stage	Outcome	Count	Share
Initial screening	Probability too low to be interesting	0	0%

Stage	Outcome	Count	Share
Orderbook check	No orderbook available	0	0%
Price discovery	No executable ask price	0	0%
Price validation	Price outside acceptable range	0	0%
Edge calculation	Edge too small after fees	0	0%
Position sizing	Calculated size rounded to zero	0	0%
Strategy rules	Another asset chosen for this window	0	0%
Passed all filters	Trade candidate	0	0%

This extreme selectivity is by design — the system only trades when every condition aligns.

Settled Outcomes

Metric	Value
Total settled trades	0
Observed win rate	N/A
Observation P&L (cents)	0

Win Rate by Entry Price

Entry Price	Trades	Wins	Win Rate
80–84 cents	0	0	—
85–89 cents	0	0	—
90–94 cents	0	0	—
95–99 cents	0	0	—

Infrastructure and Reliability

Always-On Operation

The bot runs on a dedicated cloud server (DigitalOcean, Ubuntu 24.04) managed by systemd, the standard Linux process manager. If the bot crashes for any reason, systemd automatically restarts it within seconds. The system has been designed for unattended 24/7 operation.

Continuous Deployment

Code changes pushed to the main branch automatically deploy to the production server via GitHub Actions:

1. The CI pipeline SSHes into the server
2. Pulls the latest code
3. Runs a syntax check to catch errors before they reach production
4. Restarts the bot service

This pipeline ensures rapid iteration while maintaining a safety net against broken deployments.

Complete Audit Trail

Every decision the bot makes is logged:

- **SQLite database** — Stores all positions, orders, fills, settlements, and every market evaluation with its filter stage outcome
- **Nine JSONL journal files** — Append-only logs covering scans, opportunities, rejections, trades, settlements, orders, execution events, performance summaries, and maker order fill model training data
- **Firebase dashboard** — Real-time web interface showing current positions, market evaluations, system status, and execution engine statistics

This comprehensive logging enables full after-the-fact analysis of any trade or decision.

Crash Recovery

Order identifiers are written to the database before API submission. If the bot crashes mid-order, it can reconcile its state on restart without placing duplicate orders or losing track of open positions.

Technical Appendix

For readers interested in the mathematical foundations, the full technical whitepaper provides detailed formulas and derivations. Brief summaries of the key models:

Volatility Model — Uses the Realized Kernel estimator (Barndorff-Nielsen 2008) with data-adaptive bandwidth selection to produce noise-robust volatility from high-frequency returns. Multiple estimators are blended using Mincer-Zarnowitz R^2 -weighted EMA blending, replacing fixed weights with data-driven quality scores. Includes adaptive jump detection (percentile-based thresholds per asset) and options-implied volatility integration from Deribit. An EGARCH(1,1) model with Student-t innovations runs in shadow mode, computing forecasts without affecting live decisions.

Probability Model — Computes win probability using the Normal Inverse Gaussian (NIG) distribution with per-asset fitted parameters (a, b, ν, μ), capturing both heavy tails and asymmetry. NIG dramatically outperforms Student-t on statistical fit tests. Calibration is data-driven: as settlement outcomes accumulate, the CalibrationEngine progresses from fixed logistic scaling to Platt Scaling to Beta Calibration to Isotonic Regression. A dynamic time-dependent probability cap relaxes as expiry approaches (93% at 10min+ $\rightarrow$ 99.5% at <1min).

Position Sizing — Edge-tiered sizing with drawdown-based scaling. Higher fee-adjusted edge trades get larger allocations (50% at 4%+, 35% at 2%+, 20% at 1.5%+, 10% at 1%+), with automatic de-risking during drawdowns.

Execution Model — Maker-first with three-tier post_only rejection handler: normal maker $\rightarrow$ degraded maker (1c worse) $\rightarrow$ taker IOC (with edge re-verification). Maker orders use `post_only=True` to guarantee 75% fee savings. Fill detection via Kalshi WebSocket (zero API cost). Unfilled orders escalate via in-place amendment (`amend_order()`) before falling back to cancel + IOC (`time_in_force="immediate_or_cancel"`). Queue position monitoring every ~5s enables optimal escalation timing.

Generated: N/A
